

Стандартные отклонения и нормальное распределение, часть II

В нормальном распределении мы знаем, что среднее значение и стандартное отклонение определяют определенные характеристики формы наших данных. Примеры на рисунках.

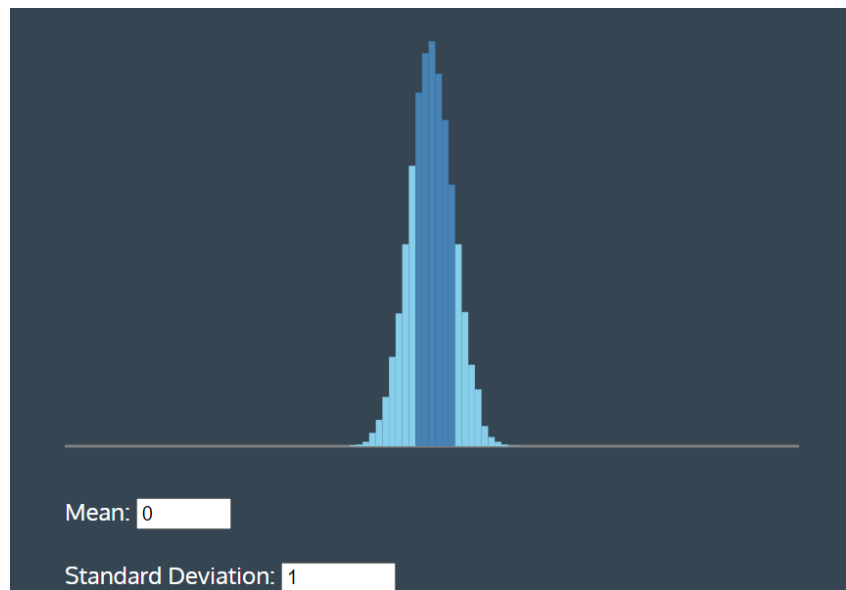


Рис.1. Среднее равно 0, стандартное отклонение равно 1 (классическое нормальное распределение)

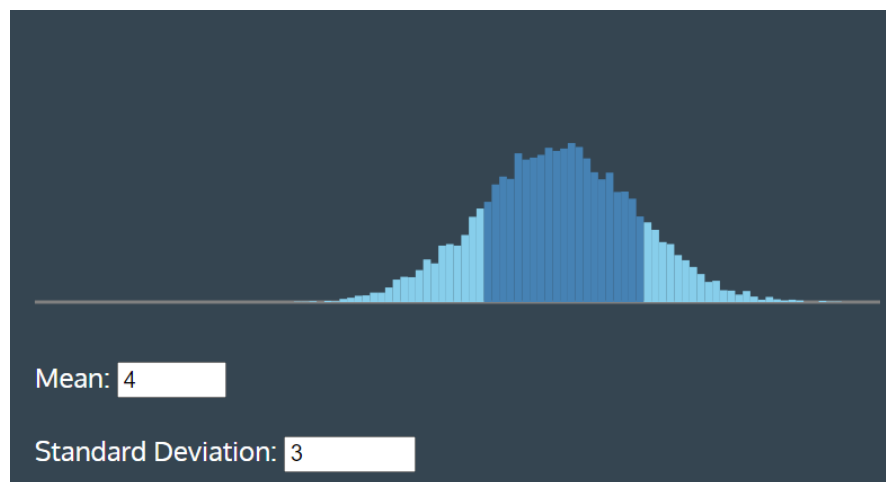


Рис.2 Среднее равно 4, стандартное отклонение равно 3

Мы видим, что стандартное отклонение влияет на «форму» нашего нормального распределения.

Предположим, что у нас есть нормальное распределение со средним значением 50 и стандартным отклонением 10. Когда мы говорим «в пределах одного стандартного отклонения от среднего», это то, что мы говорим математически:

```
lower_bound = mean - std
              = 50 - 10
              = 40

upper_bound = mean + std
```

$$\begin{aligned} &= 50 + 10 \\ &= 60 \end{aligned}$$

Оказывается, мы можем ожидать, что около 68% нашего набора данных будет лежать в пределах от 40 до 60 для этого распределения.

Как мы видели в предыдущем упражнении, независимо от того, какое среднее значение и стандартное отклонение мы выберем, 68% наших данных будут находиться в пределах ± 1 стандартное отклонение от среднего! (1σ)

Фактически, вот еще несколько полезных правил для нормальных распределений:

- 68% наших данных будут находиться в пределах ± 1 стандартное отклонение от среднего.
- 95% наших данных будут находиться в пределах ± 2 стандартных отклонения от среднего
- 99,7% наших данных будут находиться в пределах ± 3 стандартных отклонения от среднего

Задание

1. Средний балл за SAT составляет 1000, а стандартное отклонение - 100.

Вычислите сумму, которая будет на одно стандартное отклонение выше среднего, и сохраните ее в переменной `one_above`.

2. Вычислите сумму, которая будет на одно стандартное отклонение ниже среднего, и сохраните ее в переменной `one_below`.

3. Вывести в консоль значения `one_above` и `one_below`.

4. Экзамен сдали 2000 студентов. Сколько результатов будут лежать в диапазоне от `one_above` до `one_below`?

Сохраните свой ответ в переменной `one_std` и распечатайте его в терминале.

Биномиальное распределение, часть I

Известно, что некий баскетболист выполняет 30% штрафных бросков. В матче в пятницу вечером у него была возможность выполнить 10 штрафных бросков. Сколько штрафных бросков вы можете ожидать от него? Мы ожидаем $0,30 * 10 = 3$.

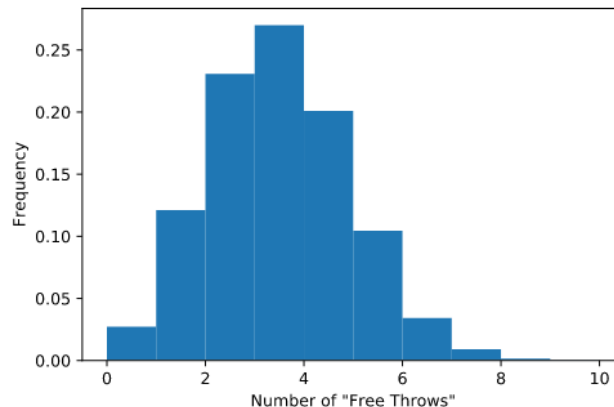
Однако на самом деле он сделал 4 штрафных броска из 10 или 40%. Это удивительно? Означает ли это, что он на самом деле его навыки лучше, чем мы думали?

Биномиальное распределение может нам помочь. Он говорит нам, насколько вероятно достижение определенного количества «успехов» с учетом вероятности успеха и количества испытаний.

В этом примере:

- Вероятность успеха 30% (он выполняет 30% штрафных бросков).
- Количество попыток - 10 (сделал 10 выстрелов).
- Количество успехов - 4 (сделал 4 выстрела).

Биномиальное распределение важно, потому что оно позволяет нам узнать, насколько вероятен определенный результат, даже если он не является ожидаемым. Из этого графика видно, что для нашего баскетболиста не так уж и маловероятно, что он получит 4 штрафных броска из 10. Однако для него маловероятно, что он получит все 10.



Есть несколько сложных формул для определения этих типов вероятностей. К счастью для нас, мы можем использовать NumPy - в частности, его способность генерировать случайные числа. Мы можем использовать эти случайные числа для моделирования распределения числовых данных, которое соответствует реальному сценарию, который мы хотим понять.

Например, предположим, что мы хотим знать разные вероятности того, что наш баскетболист сделает 1, 2, 3 и т. д. Из 10 бросков.

В NumPy есть функция для генерации биномиальных распределений: `np.random.binomial`, которую мы можем использовать для определения вероятности различных результатов.

Функция вернет количество успехов для каждого «эксперимента».

Требуются следующие аргументы:

- `N`: количество образцов или испытаний.
- `P`: вероятность успеха
- `size`: количество экспериментов

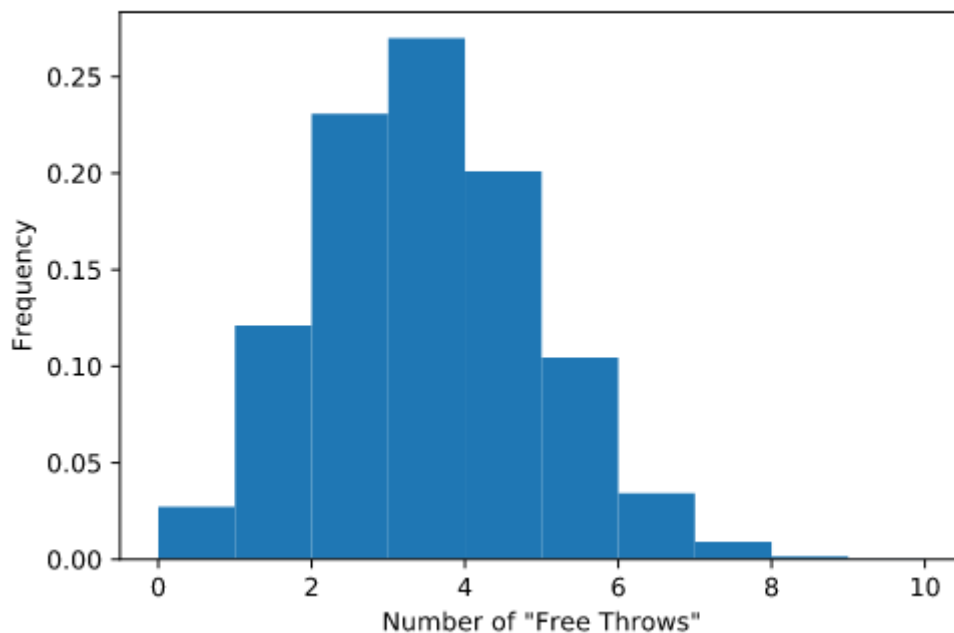
Проведем серию «экспериментов» нашего баскетболиста, сделавшего 10 бросков. Мы выбираем большое `N`, чтобы быть уверенным, что наши вероятности сходятся в правильном ответе.

```
# Let's generate 10,000 "experiments"
# N = 10 shots
# P = 0.30 (30% he'll get a free throw)

a = np.random.binomial(10, 0.30, size=10000)
```

Теперь у нас есть рекорд в 10 000 экспериментов. Мы можем использовать Matplotlib для построения результатов всех этих экспериментов:

```
plt.hist(a, range=(0, 10), bins=10, normed=True)
plt.xlabel('Number of "Free Throws"')
plt.ylabel('Frequency')
plt.show()
```



Задание

1. Человек отправляет 500 писем с просьбой о пожертвовании на их дело, с оценочной вероятностью, что 25 человек (5%) откроют письмо и нажмут, чтобы сделать пожертвование. Было проведено 10 000 экспериментов.

Используя формулу биномиального распределения, рассчитайте распределение и сохраните его в переменной email.

2. Постройте график биномиального распределения с использованием настроек гистограммы по умолчанию и отобразите его с помощью plt.show ().

Биномиальные распределения и вероятность

Вернемся к нашему исходному вопросу:

У нашего баскетболиста есть 30% шанс забросить мяч в кольцо. Он сделал 10 бросков и 4 из них были успешными, хотя мы ожидали, что он сделает успешно только 3. Какой процент шанса у него сделать эти 4 броска?

Мы можем вычислить другую вероятность, подсчитав процент экспериментов с одинаковым результатом, используя функцию pr.mean.

Помните, что среднее значение логического утверждения даст нам процент значений, удовлетворяющих нашему логическому утверждению.

Давайте посчитаем вероятность того, что он попадет 4 раза в корзину:

```
a = np.random.binomial(10, 0.30, size=10000)
np.mean(a == 4)
```

Когда мы запустим этот код, мы получим:

```
>> 0.1973
```

Помните, поскольку мы используем генератор случайных чисел, каждый раз мы будем получать немного разные результаты. При выбранном нами большом * размере расчетная вероятность должна иметь точность примерно до 2 знаков после запятой. *

Таким образом, шанс забросить 4 раза в корзину у нашего баскетболиста составляет примерно 20%.

Это говорит о том, что то, что мы наблюдали, не было таким уж невероятным. Вполне возможно, что его навыки не стали лучше, а ему просто повезло.

Задание

1. Вернемся к нашему примеру с электронной почтой. Помните, что мы отправили 500 писем, с расчетной вероятностью, что их откроют 25 человек. Было 10 000 испытаний.

Какова вероятность, что никто не откроет письмо? Сохраните результаты в переменной `no_emails`.

2. Вы недавно наняли нового сотрудника по маркетингу, который хочет провести A / B-тестирование для оценки эффективности электронных писем, чтобы узнать, как реагируют люди. Какова вероятность, что 8% или более людей откроют письмо? Сохраните результаты в переменной `b_test_emails`.
3. Выведите результаты в консоль.

Заключение

В этом уроке вы узнали, как использовать NumPy для анализа различных распределений и генерации случайных чисел для создания наборов данных. Вот что мы рассмотрели:

- Что такое гистограмма и как ее отобразить с помощью Matplotlib
- Как определить различные формы набора данных в зависимости от пиков или распределения данных
- Определение нормального распределения и то, как использовать NumPy для его генерации с помощью функций случайных чисел NumPy.
- Связь между нормальным распределением и стандартными отклонениями
- Определение биномиального распределения

Теперь вы можете использовать NumPy для анализа и построения графиков собственных наборов данных! Вы должны практиковаться в наращивании своей интуиции не только в отношении того, что говорят данные, но и в отношении того, какие выводы можно сделать из ваших наблюдений.

Задание

1. Представьте, что вы работаете в ботаническом саду и хотите посмотреть, как выглядят подсолнухи, которые вы посадили в прошлом году, чтобы увидеть, хотите ли вы посадить их больше.

Вычислите среднее значение и стандартное отклонение этого набора данных. Сохраните среднее значение в `sunflowers_mean` и стандартное отклонение в `sunflowers_std`. Набор данных загрузите в переменную `sunflowers` из CSV на основании данных:

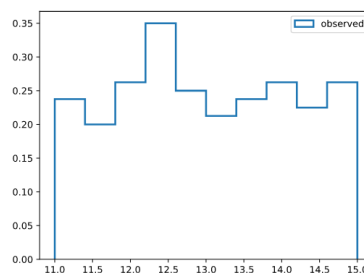
```
[ 13.89  14.7   13.    11.37  12.1   11.82  13.09  13.78
12.    12.83
   12.1   11.09  13.32  11.86  14.35  11.7   12.85  14.
13.83  12.02
   12.05  14.1   12.39  11.83  13.18  11.43  14.61  13.12
11.5    11.15
   12.64  14.07  12.44  11.21  14.31  14.62  14.67  14.55
12.72  11.19
   13.27  12.02  13.79  13.14  14.51  12.37  14.06  14.36
11.24  14.73
   13.03  13.86  13.33  11.3   12.18  11.75  14.69  14.18
14.05  12.58
   14.53  12.45  14.44  12.76  11.65  14.7   14.54  12.98
13.08  14.63
   13.22  12.59  14.62  11.37  14.55  14.7   14.89  11.45
14.74  13.78
   13.62  12.36  11.15  11.15  13.1   12.82  12.34  13.25
12.1    14.69
   14.58  14.89  14.03  12.29  12.03  11.83  13.57  11.75
13.64  12.42
   13.07  12.97  14.89  11.23  13.45  14.11  12.47  12.24
13.9    12.85
   14.65  13.54  12.76  12.41  11.61  11.77  11.44  13.85
13.02  14.61
   13.68  12.99  11.88  12.48  12.78  12.23  13.88  12.83
13.41  13.9
   12.35  12.91  11.32  13.63  11.37  12.87  11.27  12.56
11.87  12.53
   14.5   11.38  11.25  11.13  13.7   12.58  11.88  14.5
11.91  13.87
   14.38  12.04  13.61  13.1   12.96  12.3   14.54  13.66
14.86  12.42
   14.34  14.1   12.37  14.33  11.64  13.45  13.49  13.55
12.65  11.54
   12.48  12.26  13.93  11.98  12.04  14.59  11.51  12.49
13.71  11.26
   12.15  13.84  14.62  11.76  12.59  13.56  12.21  11.24
14.47  14.63
```

```
12.51 11.43 14.84 14.14 14.16 12.93 13.19 11.53
12.78 12.79]
```

2. Постройте гистограмму по нашим данным, используя код:

```
plt.hist(sunflowers,
         range=(11, 15), histtype='step', linewidth=2,
         label='observed', normed=True)
plt.legend()
plt.show()
```

3. Из гистограммы видно, что наши данные не распределяются нормально. Давайте создадим нормально распределенную выборку для сравнения с тем, что мы наблюдали.



Создайте 5000 случайных выборок с таким же средним и стандартным отклонением, как у подсолнухов. Сохраните их в `sunflowers_normal`.

4. Построить график нормального распределения, используя код:

```
plt.hist(sunflowers_normal,
         range=(11, 15), histtype='step', linewidth=2,
         label='normal', normed=True)
plt.legend()
plt.show()
```

5. Как правило, 10% посаженных подсолнухов не цветут. Мы посадили 200 и хотим знать вероятность того, что менее 20 не зацветут.

Сначала сгенерируйте 5000 биномиальных случайных чисел, которые представляют нашу ситуацию. Сохраните их в `experiments`.

6. В каком проценте экспериментов не зацвело менее 20 подсолнухов?

Сохраните свой ответ в переменной `prob`. Это приблизительная вероятность того, что менее 20 наших подсолнухов не зацветут.

7. Выведите `prob`. Возможно ли, что менее 20 наших подсолнухов не зацветут?